Embeddings & Position

Lecture 4 · Large Language Models

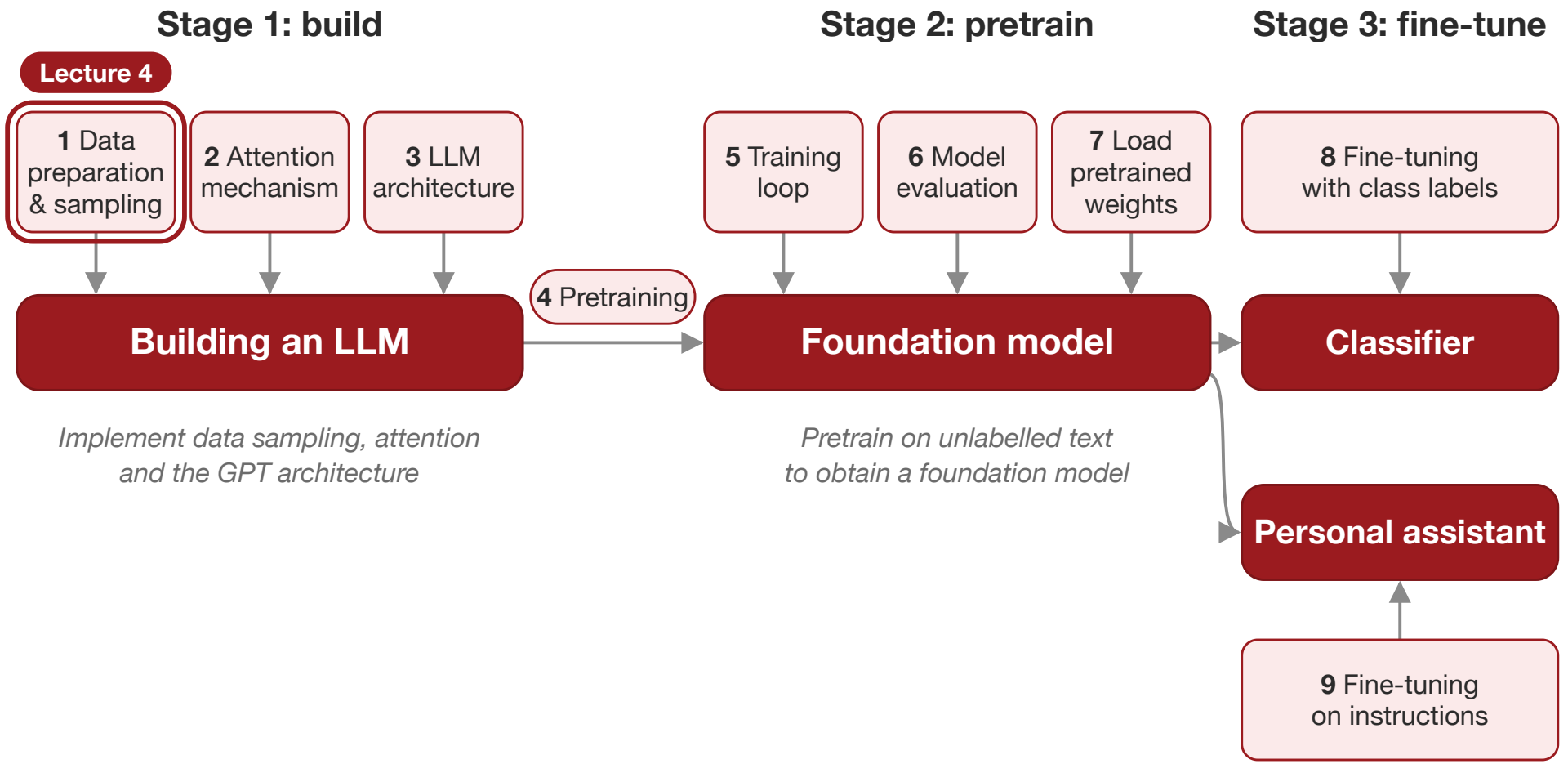
Daniel Kucharczyk
Wrocław University of Science and Technology

2026-10-23

Recap: text now arrives as a sequence of token IDs

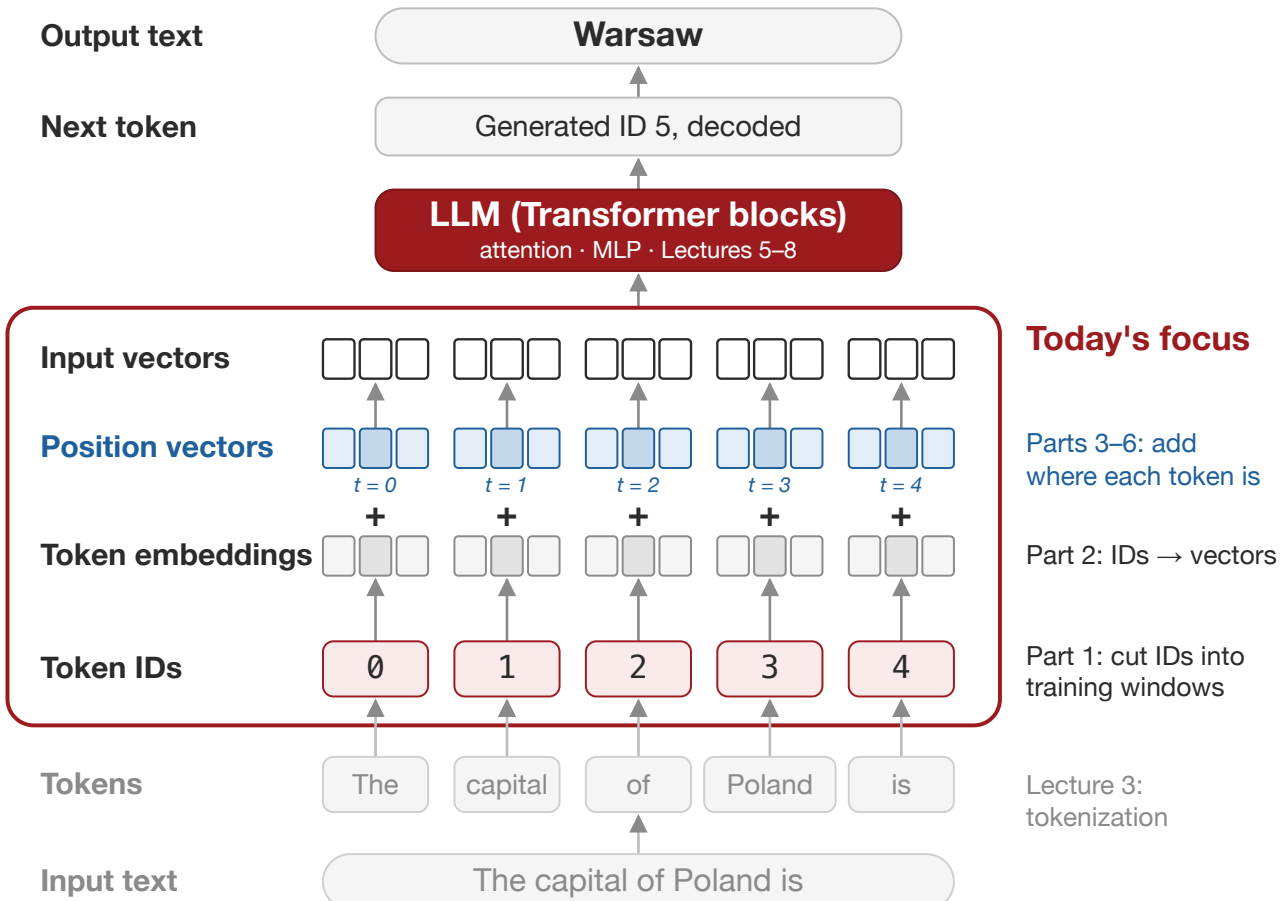
- Lecture 3: text → UTF-8 bytes → BPE pieces → IDs.
- A prompt of T tokens is a vector of IDs, shape $(T,)$.
- IDs are names: 5 is not closer to 6 than to 500.
- **Open question:** how does a model compute with names?

Lecture 4 finishes box 1: data preparation and sampling



Course roadmap with today's box outlined. Redrawn after Raschka (Raschka 2024).

Today: from token IDs to input vectors that know their position



The Lecture 3 pipeline with today's focus: IDs, token embeddings and position vectors. Pieces and IDs are illustrative.

Does `dog bites man` mean `man bites dog`?

- Same three tokens, opposite meaning.
- Bag-of-words (Lecture 1) cannot tell them apart.
- Self-attention (Lecture 5) on its own cannot either.
- Today: how a Transformer learns **where** each token is.

Agenda

1. Training pairs from a sliding window
2. From IDs to vectors: embedding tables
3. Why order must be injected
4. Absolute positions: learned and sinusoidal
5. From absolute to relative positions
6. Rotary embeddings and long contexts
7. Comparison, Lab 4 and takeaways

Learning objectives (1–4)

1. Compute the number of training samples produced by a sliding window with given context length and stride.
2. Show that an embedding lookup equals a linear layer applied to a one-hot input, and state which rows of the embedding matrix receive non-zero gradients.
3. Explain why self-attention needs positional information, and compute the parameter count of the token and learned position embeddings of a GPT model.
4. Show that $\mathrm{PE}(t) \cdot \mathrm{PE}(t+k)$ depends only on k, and that $\mathrm{PE}(t+k)$ is a rotation $R(k)$ of $\mathrm{PE}(t)$.

Learning objectives (5–8)

5. Expand the attention score of inputs with added absolute positions into its four terms and explain why it is no longer a function of the offset alone.
6. Write the attention score with a T5 relative bias and with ALiBi, and explain why ALiBi extrapolates to longer sequences.
7. Show that with rotary embeddings the score $(R_t\mathbf{q})^\top(R_s\mathbf{k})$ depends only on $\mathbf{q}$, $\mathbf{k}$ and $s-t$.
8. Compare learned absolute, sinusoidal, relative-bias and rotary encodings by where position enters, number of parameters and behaviour beyond the training length.

These objectives define the exam scope.

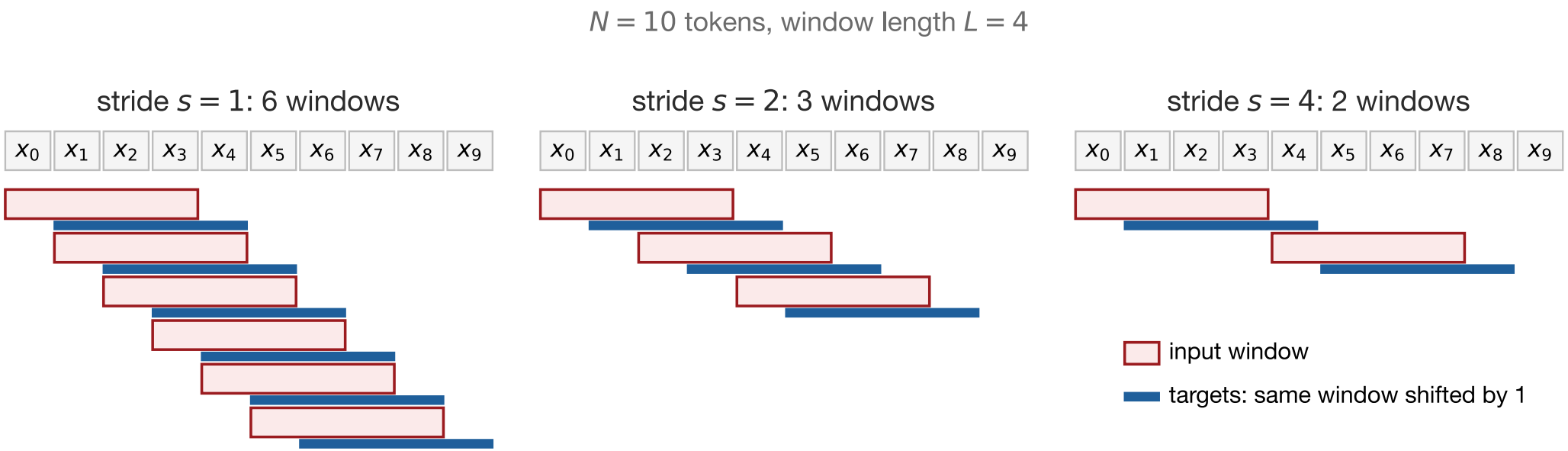
Part 1: Training pairs from a sliding window

The target sequence is the input shifted by one token

$$\text{input} = (x_i, \ldots, x_{i+L-1}), \qquad \text{target} = (x_{i+1}, \ldots, x_{i+L})$$

- One window of length L holds L predictions.
- Position j predicts x_{i+j+1} from $x_i, \ldots, x_{i+j}$.
- No labels needed: the text supervises itself.

The stride sets how much consecutive windows overlap



Windows over $x_0, \ldots, x_9$ for three strides; each target bar is its input window shifted by one token.

- $s < L$: more, strongly correlated samples; $s = L$: each token is a target about once.

The window count follows from where the last target sits

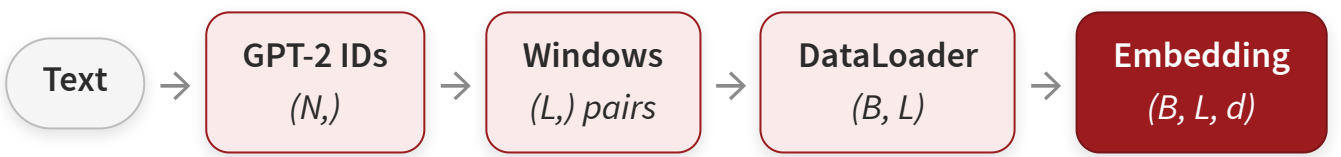
Proposition (objective 1)

Window i needs x_{i+L} , so $i \le N - L - 1$. Starts $0, s, 2s, \dots$ give

$$n_{\text{windows}} = \left\lfloor \frac{N - L - 1}{s} \right\rfloor + 1.$$

- Toy: $N = 10, L = 4, s = 2$: $\lfloor 5/2 \rfloor + 1 = 3$.
- In code: the starts of `range(0, N - L, s)`.

A batch of windows is a (B, L) tensor of token IDs



- `create_dataloader` (Lab 4.2): a `Dataset` of windows plus a `DataLoader`.
- `shuffle=True` for training only; `drop_last=True` keeps shapes equal.

Part 2: From IDs to vectors

A token ID only says which row of a matrix to read

- Each token gets a learned vector in $\mathbb{R}^d$: its **embedding**.
- All embeddings form $E \in \mathbb{R}^{V \times d}$, one row per token.
- Token i → row $E_{i,:}$; a sequence of T IDs → (T, d).
- GPT-2 small: $V = 50{,}257, d = 768$.
- Rows start random; training moves them.

Words in similar contexts get similar embeddings

- “You shall know a word by the company it keeps” (Firth 1957).
- Distributional hypothesis (Harris 1954).
- `I fed the cat` / `I fed the dog`: same targets, similar gradients.
- GPT: embeddings are learned by next-token prediction, with the rest.

Cosine similarity compares directions, not lengths

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|} \in [-1, 1]$$

- 1: same direction; 0: orthogonal; −1: opposite.
- One-hot: $\cos(\mathbf{e}_i, \mathbf{e}_j) = 0$ for every $i \neq j$.
- Rows of E scaled to length 1 give U: all similarities are $UU^\top$.

⚠ **Close is not synonymous**

`hot` and `cold` share contexts, so their vectors are close.

An embedding lookup is a one-hot vector times the matrix

Proposition (objective 2)

For the one-hot vector $\mathbf{e}_i \in \{0,1\}^V$: $\mathbf{e}_i^\top E = E_{i,:}$. For T IDs as one-hot rows of $X \in \{0,1\}^{T \times V}$: $XE \in \mathbb{R}^{T \times d}$ stacks their embeddings.

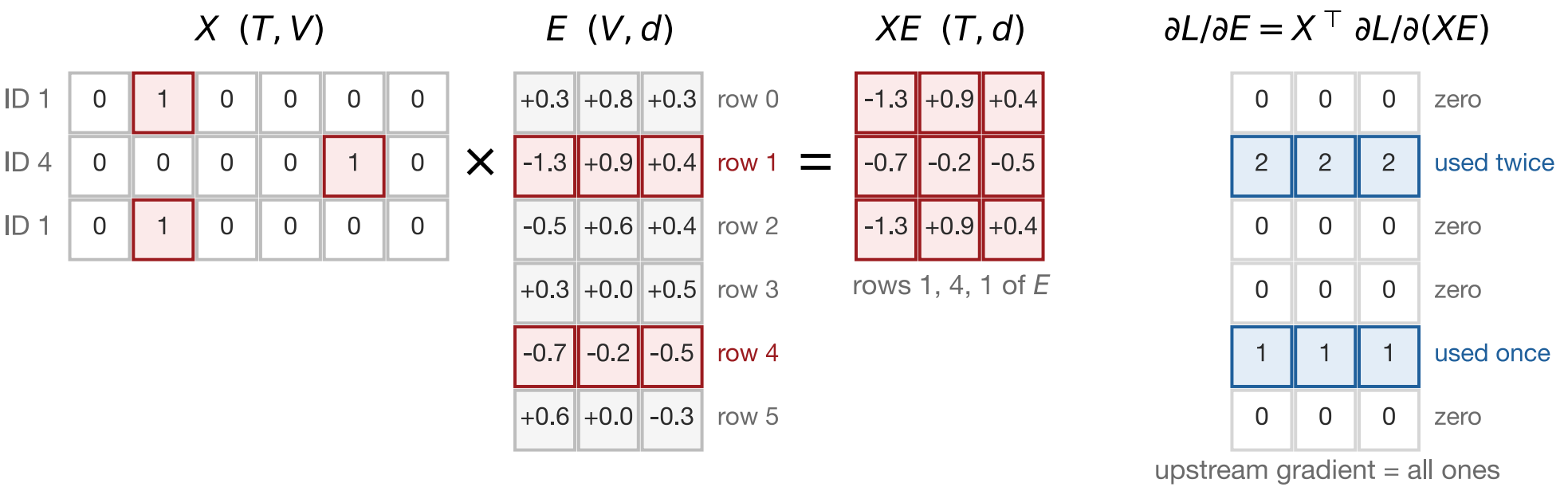
- Proof: $(\mathbf{e}_i^\top E)_j = \sum_v [v = i]\, E_{vj} = E_{ij}$.
- So an embedding layer is a linear layer on one-hot inputs.
- The lookup computes it without ever forming X.

`nn.Linear` stores the transposed matrix

Layer	`weight` shape	Computes
`nn.Embedding(V, d)`	(V, d), equal to E	rows $E_{x_t,:}$
`nn.Linear(V, d, bias=False)`	(d, V), call it W	$XW^\top$

- The outputs agree when $W = E^\top$.
- Lookup: copy T rows, $O(Td)$.
- One-hot product: $O(TVd)$; for GPT-2, $V \approx 5 \cdot 10^4$ times more work.

Only the rows that were looked up receive a gradient

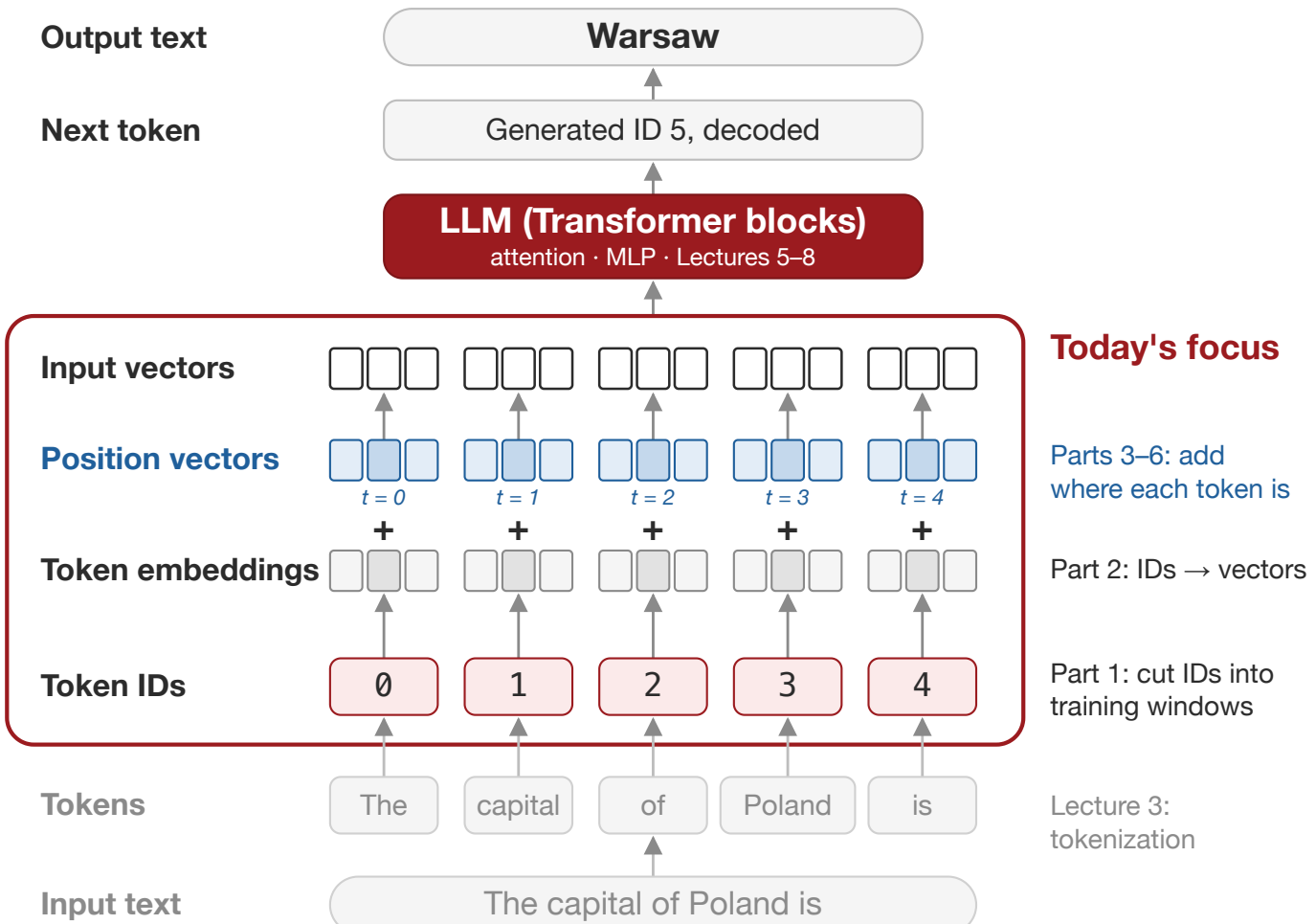


Lookup of IDs 1, 4, 1 as XE (left) and the gradient with respect to E (right), $V = 6, d = 3$.

- Token used twice: two gradients add up. Unused token: zero row.

Part 3: Why order must be injected

Parts 1–2 built the grey rows; the blue row is next



The pipeline from the start of the lecture: Parts 3–6 explain the blue position row.

Embedding vectors carry no information about position

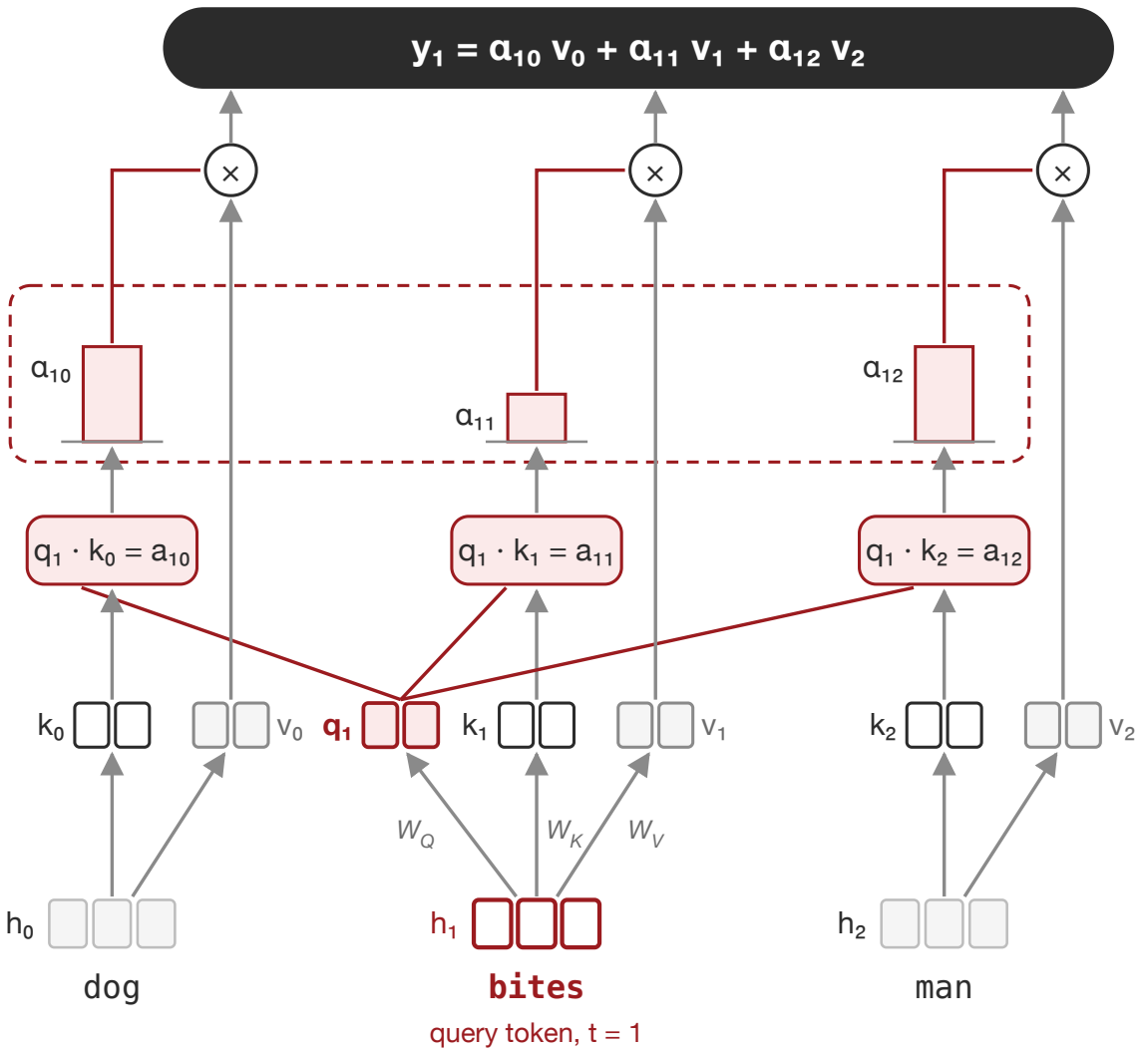
- `dog bites man` → rows $E_{\text{dog}}, E_{\text{bites}}, E_{\text{man}}$.
- `man bites dog` → the same three rows, reordered.
- The lookup acts on each position separately.
- Everything after it must recover order from somewhere.

Self-attention mixes vectors by content alone

$$\mathbf{y}_t = \sum_s \alpha_{ts}\, W_V \mathbf{h}_s\,, \qquad \alpha_{ts} = \mathrm{softmax}_s\left(\frac{\mathbf{q}_t^\top \mathbf{k}_s}{\sqrt{d}}\right)$$

- Query $\mathbf{q}_t = W_Q \mathbf{h}_t$, key $\mathbf{k}_s = W_K \mathbf{h}_s$.
- Score $a_{ts} = \mathbf{q}_t^\top \mathbf{k}_s$: contents only, no t or s.
- Preview; the derivation is in Lecture 5.

Score every key, average the values



4 · average

$y_1 = \Sigma_s \alpha_{1s} v_s$

3 · normalise (sum to 1)

$\alpha_{1s} = \text{softmax}_s(a_{1s} / \sqrt{d})$

2 · score

$a_{1s} = q_1 \cdot k_s$

1 · project

$q_1 = W_Q h_1$, $k_s = W_K h_s$, $v_s = W_V h_s$

input vectors

h_s from box 1 (token + position)

Self-attention for the query `bites`; weights illustrative. Adapted from (Raschka 2023).

Without positions, self-attention is permutation-equivariant

Proposition

For inputs $H \in \mathbb{R}^{T \times d}$ and every permutation matrix P: $\mathrm{SA}(PH) = P\,\mathrm{SA}(H)$.

- Permuting the inputs only permutes the outputs.
- `dog bites man` and `man bites dog`: same vectors, reordered.
- Proof: Lecture 5, objective 3.

A good positional encoding meets five requirements

1. **Unique:** a different code for every position.
2. **Bounded:** on the same scale as the embeddings.
3. **Deterministic:** same position, same code.
4. **Distance-aware:** offsets easy to read off.
5. **Extrapolating:** works for lengths unseen in training.

Raw position indices fail these requirements

Candidate $\mathbf{p}_t$	Problem
the number t	unbounded: $t = 1000$ dwarfs the embeddings
t/T	the same t means different things for different T
one-hot of t	$T_{\max}$ extra dimensions, no notion of distance

- **Absolute:** where am I? **Relative:** how far apart are we?

Part 4: Absolute positions

GPT-2 adds a learned vector for every position

$$\mathbf{h}_t = E_{x_t,:} + P_{t,:}, \qquad P \in \mathbb{R}^{T_{\max} \times d}$$

- P is a second `nn.Embedding`, trained together with E.
- The same token at two positions gets two different $\mathbf{h}_t$.
- Batch: $(B, T, d) + (T, d)$ broadcasts over B.
- Used by GPT-2 and BERT (Radford et al. 2019; Devlin et al. 2019).

The two embedding tables hold $(V + T_{\max})\, d$ parameters

GPT-2 small	Value
Vocabulary V	50,257
Context length $T_{\max}$	1,024
Width d	768
$(V + T_{\max})\, d$	**39,383,808**

- About 32% of GPT-2 small's 124M parameters.
- Position table alone: $1024 \cdot 768 = 786{,}432$.

A learned table has no row for position $T_{\max}$

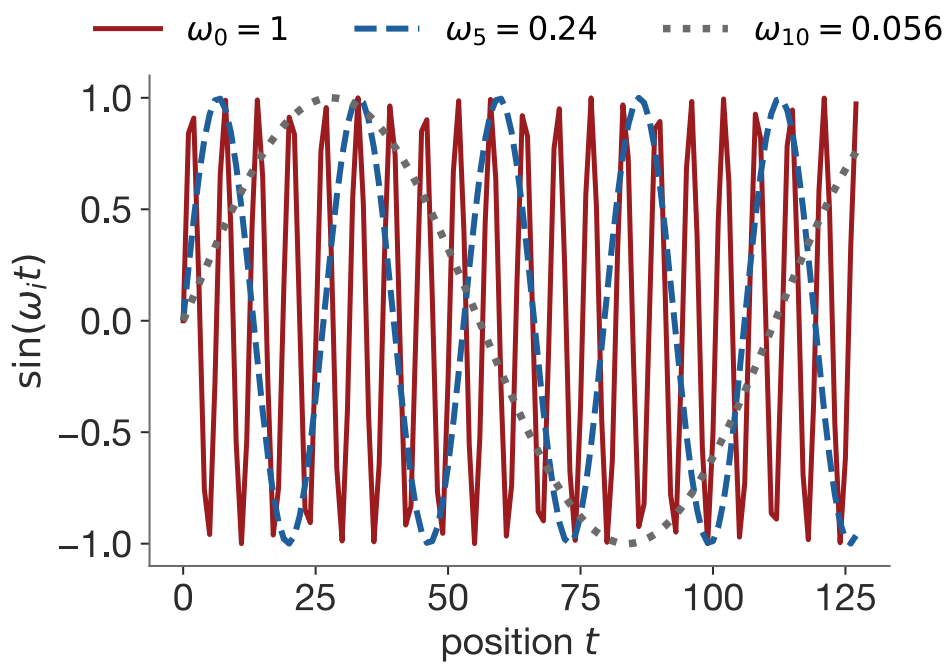
- GPT-2 at position 1024: nothing to look up.
- Each row is learned separately: nothing ties $P_{100,:}$ to $P_{101,:}$.
- Distance must be inferred from learned vectors.
- Wanted: a formula valid for every t.

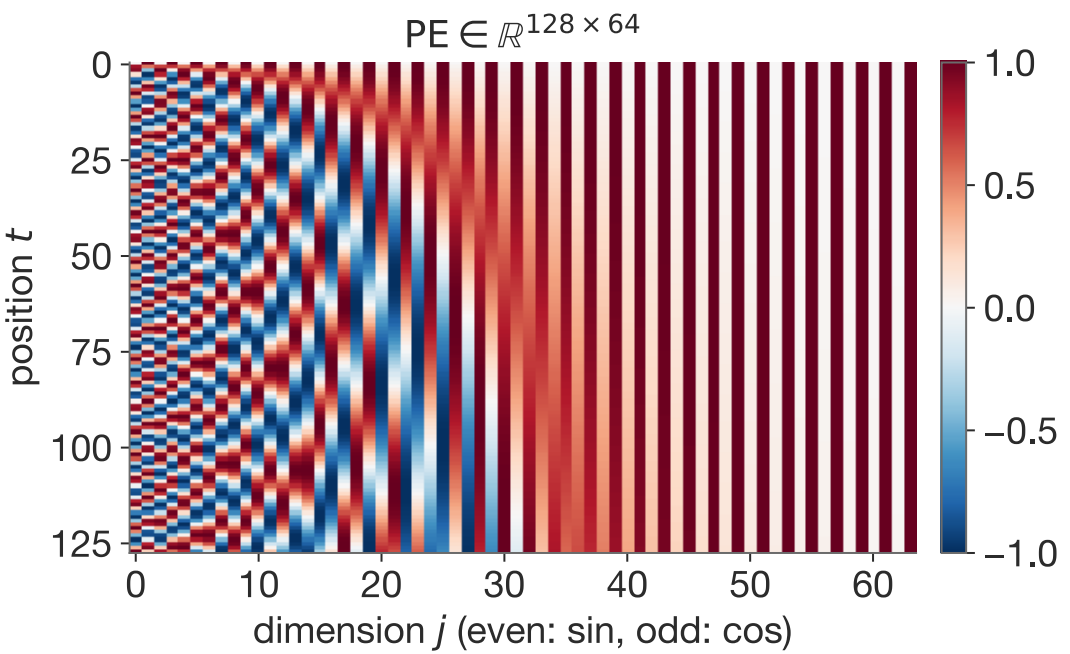
Sinusoidal encodings are a fixed function of the position

$$\mathrm{PE}(t)_{2i} = \sin(\omega_i t), \qquad \mathrm{PE}(t)_{2i+1} = \cos(\omega_i t), \qquad \omega_i = 10000^{-2i/d}$$

- t: position; $i = 0, \ldots, d/2 - 1$: index of a (sin, cos) pair.
- ω_i from 1 down to almost 10^{-4} radians per position.
- Wavelengths $2\pi/\omega_i$: from 6.3 to almost $62{,}832$ positions.
- No parameters; defined for every t (Vaswani et al. 2017).

Each (sin, cos) pair is a clock hand turning at its own speed





Sinusoidal encoding for $d = 64$: three sine dimensions (left) and the full matrix (right).

With $d = 4$, a fast and a slow pair encode each position

$\omega_0 = 1,\ \omega_1 = 10000^{-1/2} = 0.01$:

t	$\sin t$	$\cos t$	$\sin 0.01t$	$\cos 0.01t$
0	0	1	0	1
1	0.841	0.540	0.010	1.000
2	0.909	−0.416	0.020	1.000

- The fast pair separates neighbours; the slow pair separates far positions.

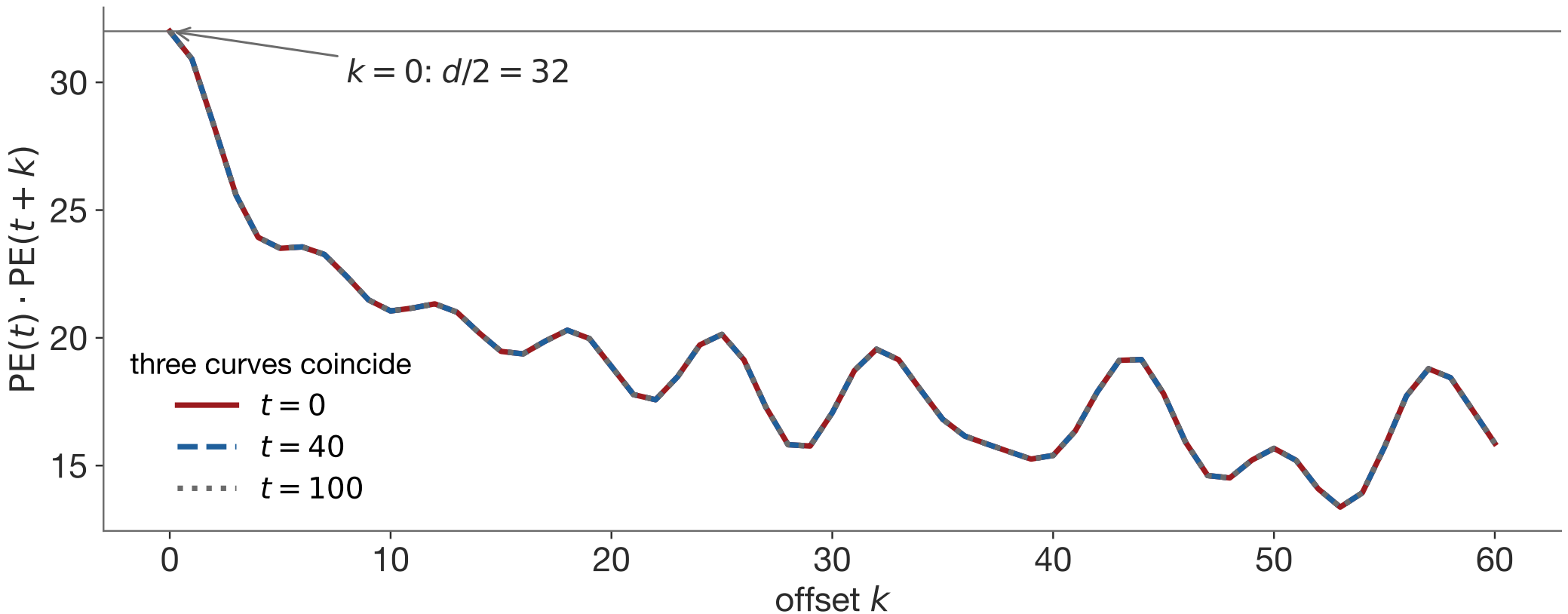
The dot product of two encodings depends only on the offset

Proposition (objective 4, part 1)

$$\mathrm{PE}(t) \cdot \mathrm{PE}(t+k) = \sum_{i=0}^{d/2-1} \cos(\omega_i k)$$

- One pair: $\sin a \sin b + \cos a \cos b = \cos(a-b)$.
- With $a = \omega_i t, b = \omega_i (t+k)$: $\cos(\omega_i k)$; t cancels.
- At $k = 0$ every term is 1: the sum is $d/2$.

All starting positions give the same similarity curve



$\mathrm{PE}(t) \cdot \mathrm{PE}(t+k)$ for $d = 64$ and three starting positions.

Shifting by k positions rotates each pair

$$\begin{pmatrix} \sin \omega_i (t+k) \\ \cos \omega_i (t+k) \end{pmatrix} = \underbrace{\begin{pmatrix} \cos \omega_i k & \sin \omega_i k \\ -\sin \omega_i k & \cos \omega_i k \end{pmatrix}}_{M_i(k)} \begin{pmatrix} \sin \omega_i t \\ \cos \omega_i t \end{pmatrix}$$

- From $\sin(a+b)$ and $\cos(a+b)$ with $a = \omega_i t, b = \omega_i k$.
- $M_i(k)$ is orthogonal with determinant 1: a rotation.
- $R(k) = \text{blockdiag}(M_0(k), \ldots, M_{d/2-1}(k))$.

Sinusoids were chosen in the hope that they extrapolate

- Learned vs sinusoidal: “nearly identical results” (Vaswani et al. 2017).
- Sinusoids “may allow the model to extrapolate” to longer inputs.
- In practice perplexity degrades beyond the training length (Press et al. 2022).
- Why? Look at what attention does with $\mathbf{x}_t + \mathbf{p}_t$.

Part 5: From absolute to relative positions

Adding positions mixes content and position in every score

$$a_{ts} = \underbrace{\mathbf{x}_t^\top W \mathbf{x}_s}_{\text{(a)}} + \underbrace{\mathbf{x}_t^\top W \mathbf{p}_s}_{\text{(b)}} + \underbrace{\mathbf{p}_t^\top W \mathbf{x}_s}_{\text{(c)}} + \underbrace{\mathbf{p}_t^\top W \mathbf{p}_s}_{\text{(d)}}$$

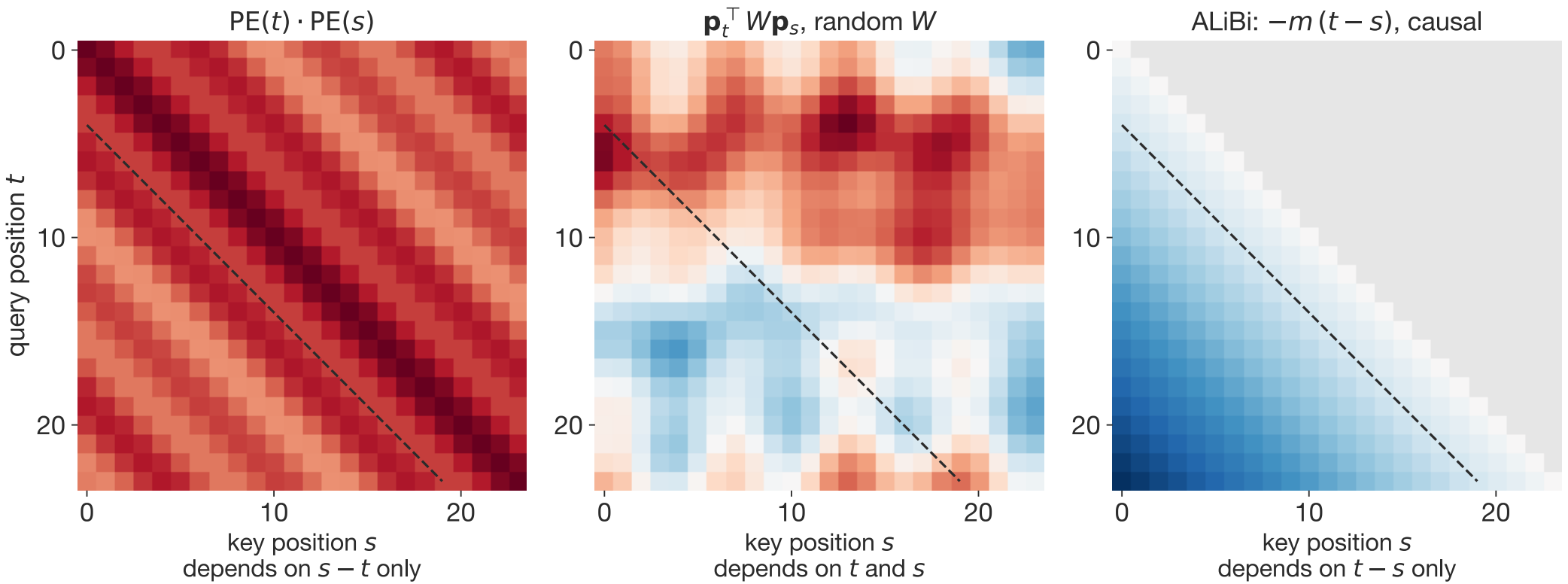
query ↓ · key →	**content $\mathbf{x}_s$**	**position $\mathbf{p}_s$**
content $\mathbf{x}_t$	(a) content–content	(b) **absolute s**
position $\mathbf{p}_t$	(c) **absolute t**	(d) position–position

Inputs $\mathbf{h}_t = \mathbf{x}_t + \mathbf{p}_t$, $W = W_Q^\top W_K$. Decomposition as in Dai et al. (2019); analysis in Ke et al. (2021).

The offset-only property does not survive the projections

- Part 4: $\mathbf{p}_t^\top \mathbf{p}_s$ depends on $s - t$ only.
- But (d) is $\mathbf{p}_t^\top W \mathbf{p}_s$ with a learned W.
- In general $R(t)^\top W R(s) \neq W R(s - t)$: t does not cancel.
- b. and (c) depend on absolute positions by construction.
- Relative patterns can be learned, but nothing enforces them.

Relative means constant along the diagonals of the score matrix

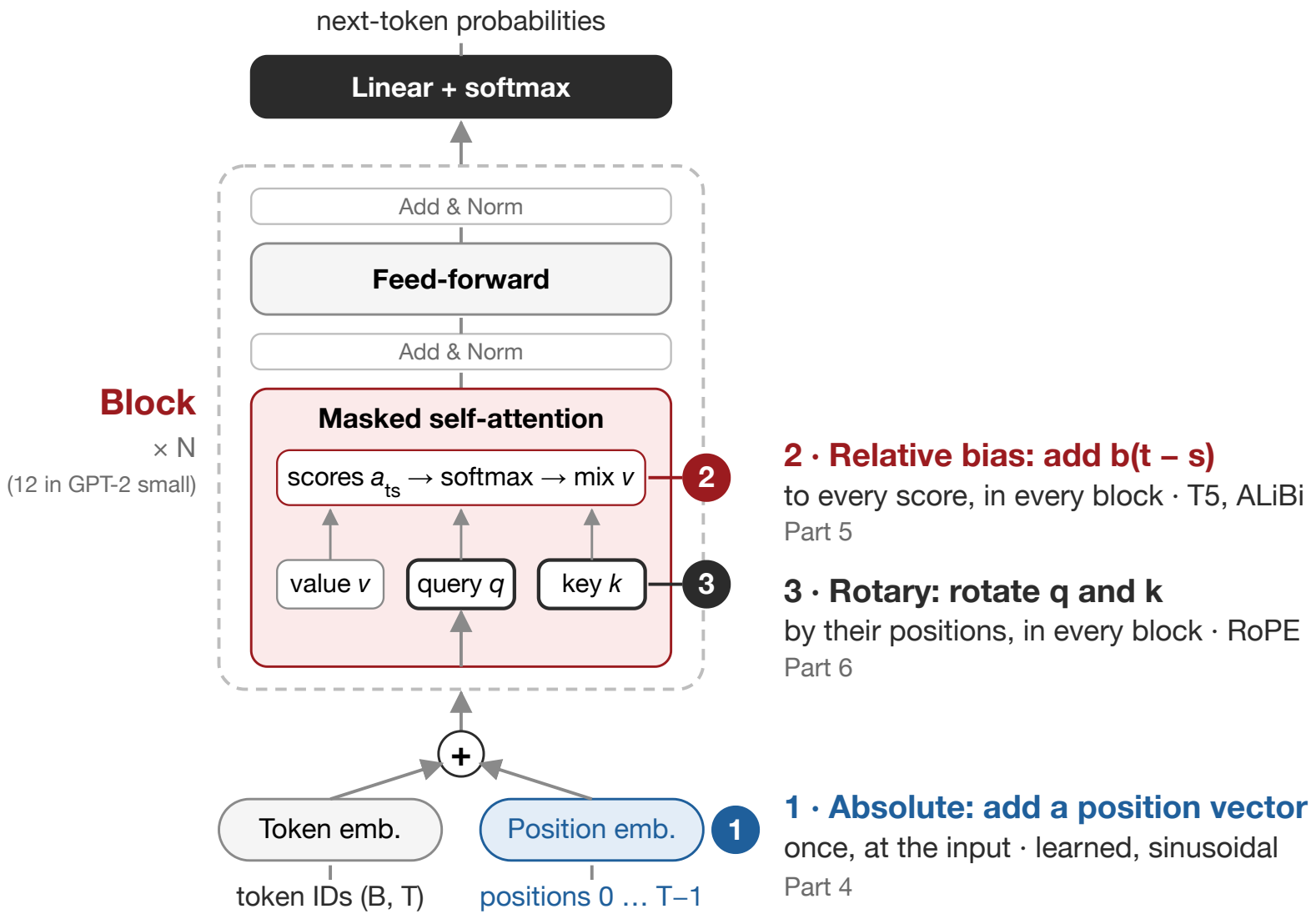


Three $T \times T$ matrices for $T = 24, d = 16$; dashed line: all pairs with $t - s = 4$.

Relative encodings put the offset directly into the score

- "Two tokens back" should mean the same at $t = 5$ and $t = 500$.
- Inputs stay pure content: $\mathbf{h}_t = E_{x_t,:}$.
- Each score gets a term that depends on $s - t$ only.
- Absolute encodes position t, once at the input; relative encodes offset $s - t$, in every layer.

Position can enter GPT at three places



A decoder-only Transformer (GPT); markers 1–3 show where each family of positional encodings acts.

Shaw et al. learn one vector per clipped offset

$$a_{ts} = \frac{\mathbf{q}_t^\top \left(\mathbf{k}_s + \mathbf{r}_{\mathrm{clip}(s-t,\, K)} \right)}{\sqrt{d}}$$

- $\mathbf{r}_{-K}, \ldots, \mathbf{r}_K$: $2K + 1$ learned vectors.
- Offsets beyond $\pm K$ share the end vectors.
- Clipping makes every length readable (Shaw et al. 2018).

T5 adds one learned scalar per distance bucket

$$a_{ts} = \mathbf{q}_t^\top \mathbf{k}_s + b_h\left(\text{bucket}(t-s)\right)$$

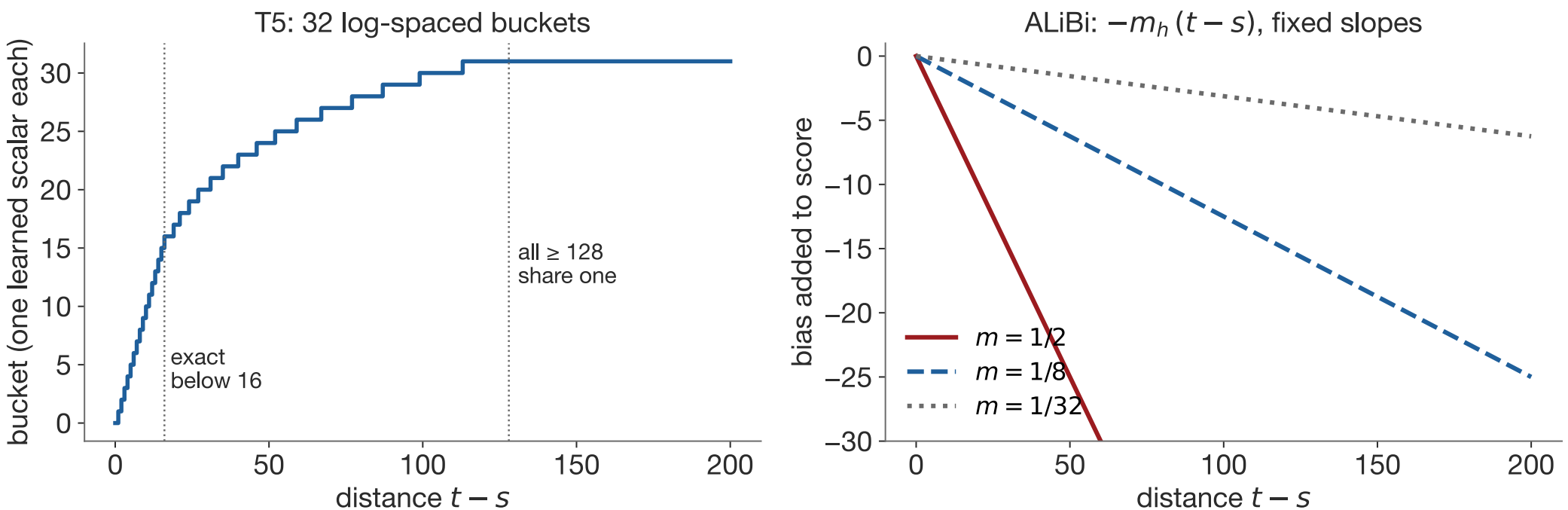
- One scalar per bucket and head, shared across layers.
- 32 buckets: exact for small distances, log-spaced up to 128.
- All distances of 128 or more share one bucket (Raffel et al. 2020).

ALiBi subtracts a fixed penalty proportional to distance

$$a_{ts} = \frac{\mathbf{q}_t^\top \mathbf{k}_s}{\sqrt{d}} - m_h\,(t - s), \qquad s \le t$$

- No positional embeddings at all and no parameters.
- Slopes $m_h = 2^{-8h/H} : \frac{1}{2}, \frac{1}{4}, \ldots, \frac{1}{256}$ for $H = 8$ heads.
- Steep heads attend locally; shallow heads look far (Press et al. 2022).

T5 and ALiBi both bias the score by distance only



Left: T5 bucket index against distance. Right: ALiBi penalties for three head slopes.

ALiBi extrapolates because its bias is defined for every distance

- A longer input brings longer distances, and $-m_h(t-s)$ covers them.
- Far tokens get large penalties: scores stay in a familiar range.
- Trained on 1024 tokens, tested on 2048: as good as sinusoidal trained on 2048 (Press et al. 2022).
- Learned absolute: no row; sinusoidal: unseen absolute positions.

The ALiBi bias is a fixed lower-triangular matrix per head

```python
import torch

T, H = 5, 4
slopes = 2.0 ** (-8 * torch.arange(1, H + 1) / H)  # (H,)
t = torch.arange(T)
dist = (t[:, None] - t[None, :]).clamp(min=0)      # (T, T): t - s
bias = -slopes[:, None, None] * dist               # (H, T, T)
future = t[None, :] > t[:, None]                   # (T, T): s > t
bias = bias.masked_fill(future, float("-inf"))
print(slopes)
print(bias[0])
```

```
tensor([0.2500, 0.0625, 0.0156, 0.0039])
tensor([[-0.0000,    -inf,    -inf,    -inf,    -inf],
        [-0.2500, -0.0000,    -inf,    -inf,    -inf],
        [-0.5000, -0.2500, -0.0000,    -inf,    -inf],
        [-0.7500, -0.5000, -0.2500, -0.0000,    -inf],
        [-1.0000, -0.7500, -0.5000, -0.2500, -0.0000]])
```

Part 6: Rotary embeddings and long contexts

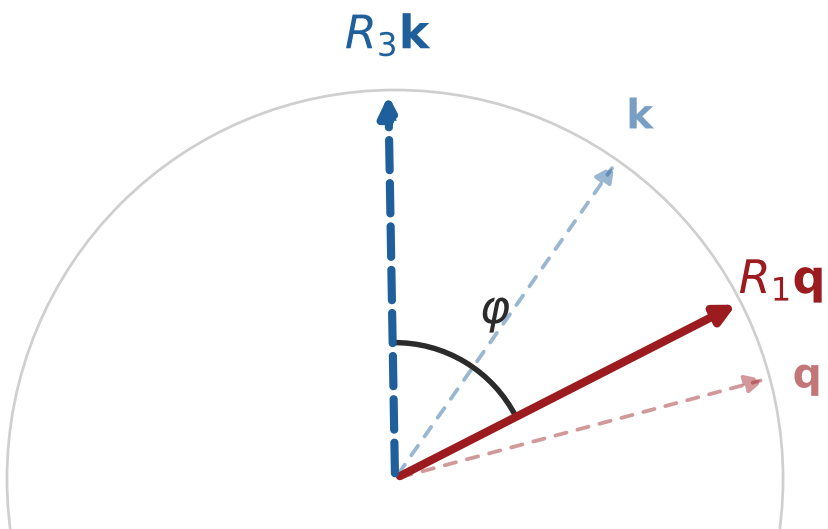
RoPE rotates queries and keys instead of changing the input

$$\rho(\phi) = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}, \qquad R_t = \operatorname{blockdiag}\left(\rho(\omega_0 t), \dots\right)$$

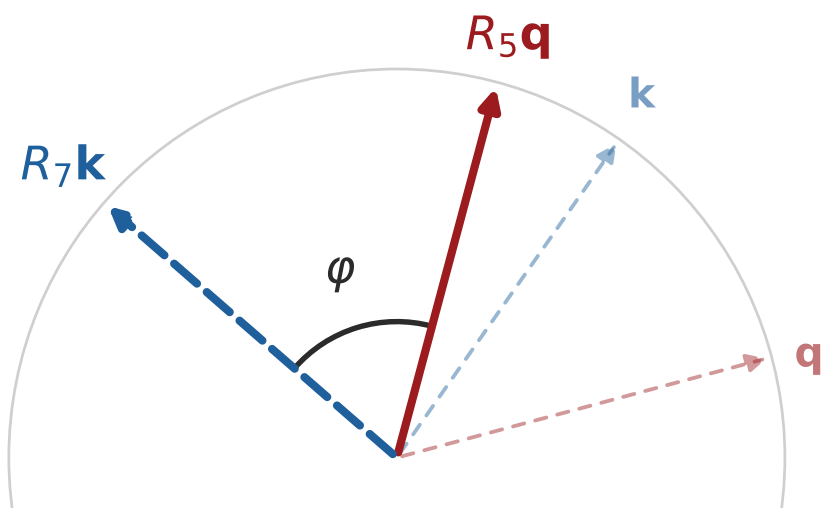
- Query at t: $R_t \mathbf{q}_t$; key at s: $R_s \mathbf{k}_s$; values unchanged.
- Pair i of each vector turns by angle $\omega_i t$.
- Same frequencies $\omega_i = 10000^{-2i/d}$ as the sinusoids (Su et al. 2021).

Rotating both vectors keeps the angle between them

$t = 1$, $s = 3$: offset $s - t = 2$



$t = 5$, $s = 7$: offset $s - t = 2$



One coordinate pair, $\omega = 12°$ per position. Same offset at two places: same angle φ, same score.

Rotary attention scores depend only on the offset

Proposition (objective 7)

$$(R_t\mathbf{q})^\top (R_s\mathbf{k}) = \mathbf{q}^\top R_t^\top R_s\,\mathbf{k} = \mathbf{q}^\top R_{s-t}\,\mathbf{k}$$

- Rotations: $\rho(\alpha)^\top = \rho(-\alpha),\ \ \rho(\alpha)\rho(\beta) = \rho(\alpha+\beta)$.
- Blockwise: $\rho(-\omega_i t)\,\rho(\omega_i s) = \rho(\omega_i(s-t))$.
- Content enters through $\mathbf{q}, \mathbf{k}$; position only through $s - t$.

RoPE keeps one term and adds no parameters

- Added encodings: four terms, two of them absolute (Part 5).
- RoPE: one term, $\mathbf{q}^\top R_{s-t}\ \mathbf{k}$, multiplicative in position.
- Applied in every layer; no learned position parameters.
- Norms preserved: $\|R_t \mathbf{q}\| = \|\mathbf{q}\|$.

Shifting both positions leaves the rotary score unchanged

```python
import torch

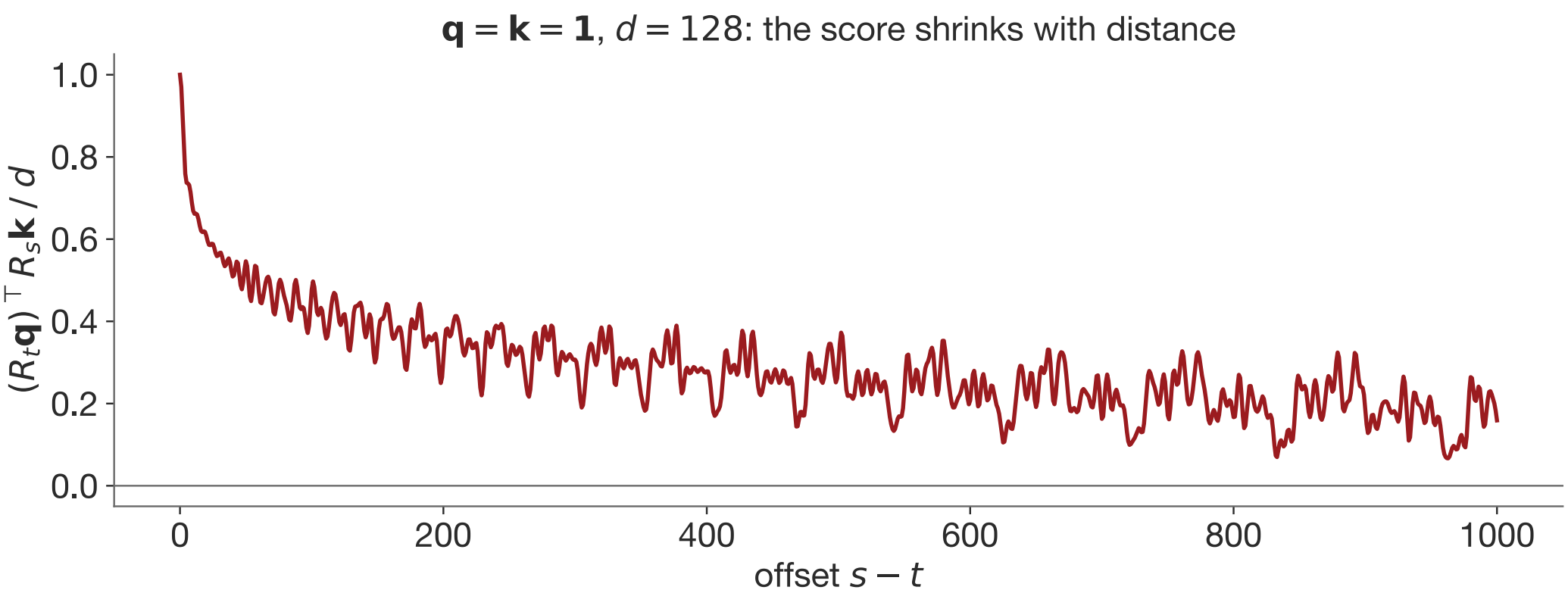
torch.manual_seed(0)
d = 8
w = 10000 ** (-torch.arange(0, d, 2, dtype=torch.float64) / d)  # (d/2,)

def rope(x, t):  # x: (d,), rotate pair i by w_i * t
    c, s = torch.cos(w * t), torch.sin(w * t)
    x1, x2 = x[0::2], x[1::2]
    return torch.stack([x1 * c - x2 * s, x1 * s + x2 * c], -1).flatten()

q, k = torch.randn(d, dtype=torch.float64), torch.randn(d, dtype=torch.float64)
for t, s in [(0, 3), (10, 13), (500, 503)]:
    print(t, s, round((rope(q, t) @ rope(k, s)).item(), 10))
```

```
0 3 1.5850570246
10 13 1.5850570246
500 503 1.5850570246
```

Rotary scores shrink with distance on average



Score of $\mathbf{q} = \mathbf{k} = \mathbf{1}$ under RoPE, $d = 128$, divided by d.

RoPE has become the default in open LLMs

Model	Positional scheme
GPT-2 (2019)	learned absolute
T5 (2020)	relative bias
BLOOM (2022)	ALiBi
LLaMA (2023), Llama 3 (2024)	RoPE

- Llama 3 raises the base from 10,000 to 500,000 for longer contexts.

Radford et al. (2019); Raffel et al. (2020); BigScience Workshop (2022); Touvron et al. (2023); Grattafiori et al. (2024).

Beyond the training length, RoPE meets unseen angles

- Trained on length L: pair i only saw angles up to $\omega_i L$.
- Slow pairs never complete a turn, so longer offsets are new.
- Position interpolation: use $t \cdot L/L'$, then fine-tune briefly.
- LLaMA extended to 32,768 tokens in at most 1000 steps (Chen et al. 2023).
- Refinements, e.g. YaRN, rescale frequencies unevenly (Peng et al. 2024).

Causal models can infer position without any encoding

- Causal mask: token t sees exactly $t + 1$ tokens.
- This breaks permutation-equivariance.
- GPT-style LMs without encodings remain competitive (Haviv et al. 2022).
- Small decoders: NoPE generalised to longer inputs best (Kazemnejad et al. 2023).
- Not the default at scale; an open research question.

Part 7: Comparison, Lab 4 and takeaways

Four families differ in where position enters

Scheme	Enters	Params	Longer inputs
Learned	input	$T_{\max} d$	undefined
Sinusoidal	input	0	degrades
T5 / ALiBi	all scores	$32H$ / 0	saturates / good
RoPE	all $\mathbf{q}, \mathbf{k}$	0	interpolate

Lab 4 turns Parts 1, 2 and 4 into code

- 4.1 (Part 2) Lookup equals one-hot × matrix; sparse gradients.
- 4.2 (Part 1) `create_dataloader`; check the window count.
- 4.3 `GPTEmbedding`; parameters and the permutation test.
- 4.4 Sinusoidal heatmap, dot product, $R(k)$ in `float64`.

Lab 4: Sampling & Embedding Pipeline · Mon 26 Oct 2026 · List 2 due Sun 8 Nov 2026, 23:59

Key takeaways

- **Obj. 1:** $\lfloor (N - L - 1)/s \rfloor + 1$ windows.
- **Obj. 2:** lookup $= \mathbf{e}_i^\top E$; only looked-up rows get gradient.
- **Obj. 3:** attention is order-blind; tables cost $(V + T_{\max})\, d$.
- **Obj. 4:** $\mathrm{PE}(t) \cdot \mathrm{PE}(t+k) = \sum_i \cos \omega_i k$;
  $\mathrm{PE}(t+k) = R(k)\, \mathrm{PE}(t)$.
- **Obj. 5–6:** added positions give 4 terms; T5 and ALiBi bias by distance.
- **Obj. 7–8:** $R_t^\top R_s = R_{s-t}$; compare where position enters.

Mandatory resources

- **BLLM Ch 2.6–2.8:** Working with Text Data (Raschka 2024).
- Vaswani et al. (2017): Attention Is All You Need, Section 3.5 (Vaswani et al. 2017).
- Su et al. (2021): RoFormer, Section 3 (Su et al. 2021).

Supporting resources: relative positions

- Shaw et al. (2018): Relative Position Representations (Shaw et al. 2018).
- Dai et al. (2019): Transformer-XL, Section 3.3 (Dai et al. 2019).
- Raffel et al. (2020): T5, Section 2.1 (Raffel et al. 2020).
- Press et al. (2022): ALiBi (Press et al. 2022).
- Ke et al. (2021): Rethinking Positional Encoding (Ke et al. 2021).

Supporting resources: long contexts and word2vec

- Chen et al. (2023): Position Interpolation (Chen et al. 2023).
- Kazemnejad et al. (2023): Positional Encoding and Length Generalization (Kazemnejad et al. 2023).
- Mikolov et al. (2013): word2vec, for optional Task 4.5 (Mikolov et al. 2013).

References

BigScience Workshop. 2022. *BLOOM: A 176B-Parameter Open-Access Multilingual Language Model*. https://arxiv.org/abs/2211.05100.

Chen, Shouyuan, Sherman Wong, Liangjian Chen, and Yuandong Tian. 2023. *Extending Context Window of Large Language Models via Positional Interpolation*. https://arxiv.org/abs/2306.15595.

Dai, Zihang, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. 2019. "Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context." *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2978–88. https://arxiv.org/abs/1901.02860.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding." *Proceedings of NAACL-HLT*, 4171–86. https://arxiv.org/abs/1810.04805.

Firth, John R. 1957. "A Synopsis of Linguistic Theory, 1930–1955." In *Studies in Linguistic Analysis*. Blackwell.

Grattafiori, Aaron et al. 2024. *The Llama 3 Herd of Models*. https://arxiv.org/abs/2407.21783.

Harris, Zellig S. 1954. "Distributional Structure." *Word* 10 (2–3): 146–62.

Haviv, Adi, Ori Ram, Ofir Press, Peter Izsak, and Omer Levy. 2022. "Transformer Language Models Without Positional Encodings Still Learn Positional Information." *Findings of the Association for Computational Linguistics: EMNLP 2022*, 1382–90. https://arxiv.org/abs/2203.16634.

Kazemnejad, Amirhossein, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. 2023. “The Impact of Positional Encoding on Length Generalization in Transformers.” *Advances in Neural Information Processing Systems* 36. https://arxiv.org/abs/2305.19466.

Ke, Guolin, Di He, and Tie-Yan Liu. 2021. “Rethinking Positional Encoding in Language Pre-Training.” *International Conference on Learning Representations*. https://arxiv.org/abs/2006.15595.

Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. *Efficient Estimation of Word Representations in Vector Space*. https://arxiv.org/abs/1301.3781.

Peng, Bowen, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. 2024. “YaRN: Efficient Context Window Extension of Large Language Models.” *International Conference on Learning Representations*. https://arxiv.org/abs/2309.00071.

Press, Ofir, Noah A. Smith, and Mike Lewis. 2022. “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation.” *International Conference on Learning Representations*. https://arxiv.org/abs/2108.12409.

Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models Are Unsupervised Multitask Learners*. OpenAI.

Raffel, Colin, Noam Shazeer, Adam Roberts, et al. 2020. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.” *Journal of Machine Learning Research* 21 (140): 1–67. https://arxiv.org/abs/1910.10683.

Raschka, Sebastian. 2023. *Understanding and Coding the Self-Attention Mechanism of Large Language Models from Scratch*. Blog post. https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html.

Raschka, Sebastian. 2024. *Build a Large Language Model (from Scratch)*. Manning. https://www.manning.com/books/build-a-large-language-model-from-scratch.

Shaw, Peter, Jakob Uszkoreit, and Ashish Vaswani. 2018. “Self-Attention with Relative Position Representations.” *Proceedings of NAACL-HLT*, 464–68. https://arxiv.org/abs/1803.02155.

Su, Jianlin, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. https://arxiv.org/abs/2104.09864.

Touvron, Hugo, Thibaut Lavril, Gautier Izacard, et al. 2023. *LLaMA: Open and Efficient Foundation Language Models*. https://arxiv.org/abs/2302.13971.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. “Attention Is All You Need.” *Advances in Neural Information Processing Systems* 30. https://arxiv.org/abs/1706.03762.

Questions?

Daniel Kucharczyk

- Wrocław University of Science and Technology

- Course repository
- Materials and submissions: ePortal
- Next: **Attention I: From Soft Lookup to Self-Attention** (Fri 6 Nov 2026).